

(a)

Neural Networks

neural networks are networks consisting of layers of neurons connected together

in Lab 3, you will implement, train, and test a simple 2-layer neural network for an image classification task

modern AI runs on deep neural networks, which look very much like the neural net you will implement but they have 10's to 100's of layers

→ e.g. the large language models (LLMs) that run Chat GPT, Claude, etc.

a single neuron ^{in a neural net} mimics ~~the~~ a neuron in the human brain → it has a bunch of inputs and "fires" when those inputs have certain values

(show neuron figure from lab assignment)

each neuron has ^a weight on each input, which is set based on ^a training process, once weights are trained, a neuron takes a weighted sum of the inputs (plus a bias) and then applies an activation function

mathematically, the neuron computes

(b)

$$Z = b + \sum w_i x_i \quad \text{over its inputs } i$$

and then the neuron output is given by $a(z)$,

where a is the activation function

common activation function is the sigmoid function

$$\sigma(z) = \frac{1}{1 + e^{-z}}$$

(show sigmoid function figure from the assignment)

an entire 2-layer network for classification

has an n to m input layer ~~and~~ ^{connected to} an m to o

output layer, where n is the number of input values (pixels in the assignment) and o is the number of categories objects are classified into

each of the ~~number of~~ outputs from the input layer (the m values) can be viewed as a feature that aids in classifying the input object

once a network is trained (all bias, weight values set), evaluating an input involves simply propagating the values through the network to produce the o output values each output value can be considered as the probability that the evaluated input's ^{proper} classification is the corresponding output

talk about the digit classification problem on the assignment and ~~start~~ running the training/testing program / (use lab assignment to aid this discussion)

©

How is a neural network trained?

most common training method: gradient descent based backpropagation

Start w/ a labeled training set (labels are human-generated correct values)

start w/ random weights, biases

for each training input, ^{do a forward propagation through network to calculate outputs, then} calculate a loss function (difference between correct outputs and outputs generated from this input)

we want to minimize loss $\rightarrow$ $\therefore$ calculate derivative (gradient) of loss wrt each weight, bias in network

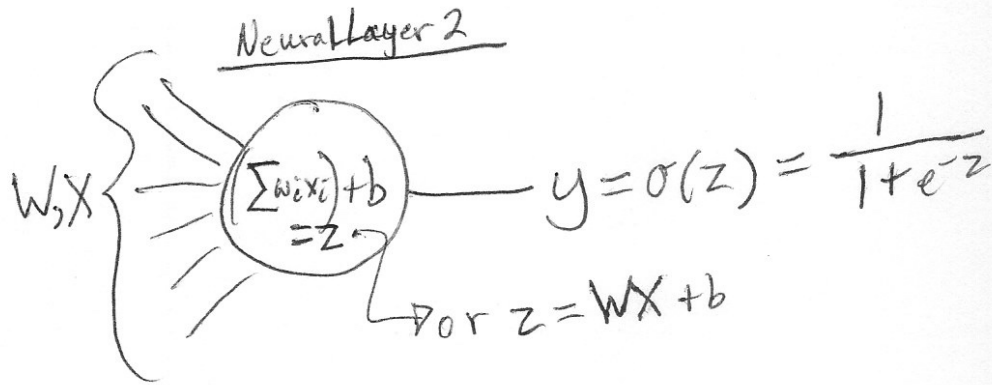
take weight (or bias) minus gradient $\rightarrow$ this is gradient descent, which moves values toward the minimum loss

to do this, need to know gradients of all biases, weights in network, this involves calculating loss gradients at output layer and propagating them backward using the derivative chain rule

show derivation for weights, biases at output layer

Calculating Gradients for Backpropagation (d)

We want $\frac{dL}{dw}$, $\frac{db}{dw}$ to minimize loss wrt w, b



$$L = \text{loss} = (y - t)^2$$
$$= y^2 - 2ty + t^2$$
$$\frac{dL}{dy} = 2y - 2t$$

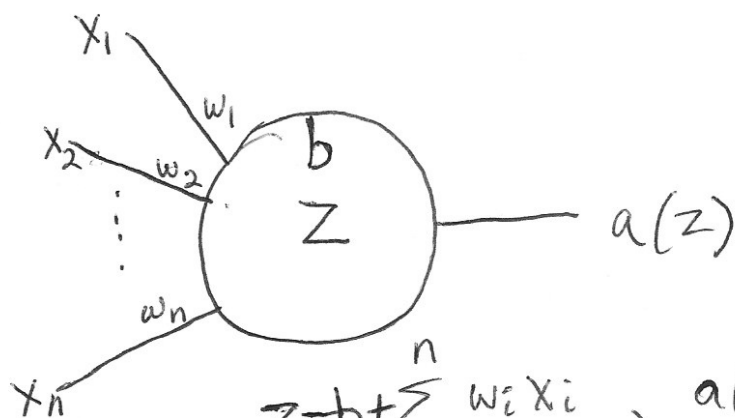
$$\frac{dy}{dz} = \sigma(z)(1 - \sigma(z))$$

$$\frac{dL}{dz} = \frac{dL}{dy} \cdot \frac{dy}{dz} = 2(y - t) \cdot \sigma(z)(1 - \sigma(z))$$

$$\frac{dL}{db} = \frac{dL}{dz} \cdot \frac{dz}{db} = 2(y - t) \cdot \sigma(z)(1 - \sigma(z)) \cdot 1$$

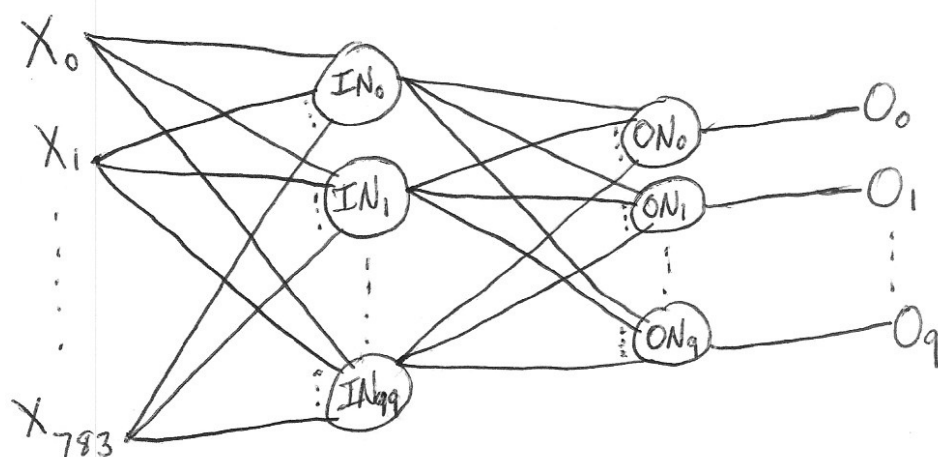
$$\frac{dL}{dw} = \frac{dL}{dz} \cdot \frac{dz}{dw} = 2(y - t) \cdot \sigma(z)(1 - \sigma(z)) \cdot x^T$$

Now, show results of training, testing program



$$z = b + \sum_{i=1}^n w_i x_i, \quad a(z) = \text{activation function}$$

operation of a neuron.



2-stage neural network for digit classification